

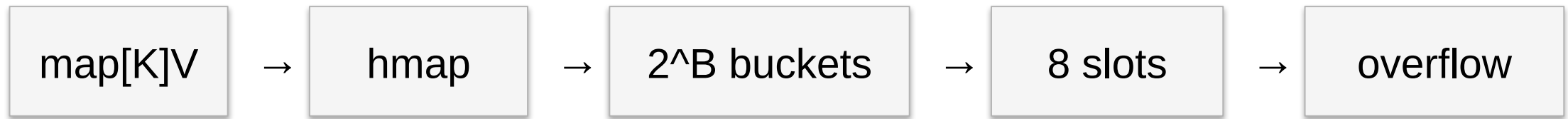
Внутреннее устройство Go map — старая реализация

Go 1.23 и раньше • bucket-based hash map • без Swiss Tables

hash → bucket[8] → tophash → key equality → overflow

Важно: это детали runtime-реализации, а не гарантии спецификации языка.

Главная ментальная модель



Старая Go map = hash table с bucket-ами по 8 элементов и цепочками overflow bucket.

hmap — верхний уровень

```
type hmap struct {  
    count      int  
    flags      uint8  
    B           uint8  
    noverflow  uint16  
    hash0      uint32  
  
    buckets    unsafe.Pointer  
    oldbuckets unsafe.Pointer  
    nevacuate  uintptr  
  
    extra *mapextra  
}
```

- count — число живых элементов, то есть len(m)
- B — $\log_2$ числа основных bucket-ов
- hash0 — seed хеширования
- buckets — текущая таблица
- oldbuckets + nevacuate — состояние постепенного grow

В и выбор bucket

Количество основных bucket-ов:

$$\text{bucketCount} = 2^B$$

Выбор bucket:

$$\text{bucket} = \text{hash} \& (2^B - 1)$$

Пример: $B = 3 \rightarrow 8$ bucket-ов. Младшие 3 бита hash задают индекс 0..7.

bmap — один bucket

```
type bmap struct {  
    tophash [8]uint8  
  
    // физически дальше в памяти:  
    // keys[8]  
    // values[8]  
    // overflow *bmap  
}
```

tophash[8]

key0 key1 ... key7

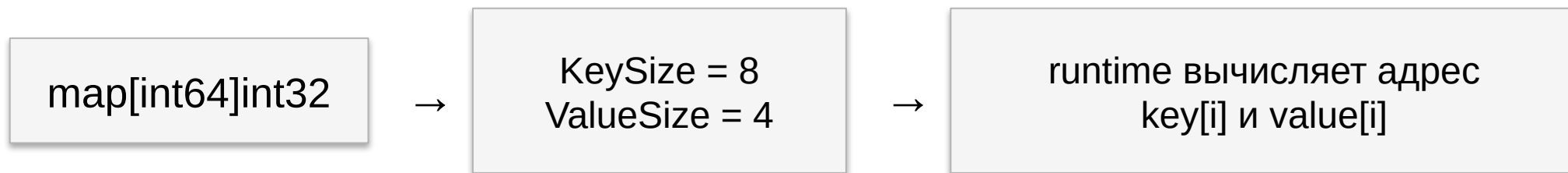
val0 val1 ... val7

overflow *bmap

keys и values лежат блоками — меньше padding и лучше layout.

Почему compiler/runtime знают layout

В bmap явно есть только tophash. Но compiler для конкретного `map[K]V` знает размеры K и V.



```
key[i] = bucketBase + dataOffset + i*KeySize  
value[i] = bucketBase + dataOffset + 8*KeySize + i*ValueSize
```

tophash — зачем он нужен

Hash одного key используется двумя способами:

HIGH 8 bits
→ tophash

полный hash(key)

LOW B bits
→ bucket index

tophash — маленький fingerprint hash. Он не доказывает равенство ключей, а только отбирает кандидатов.

tophash: конкретный пример

```
hash("alice") = 10110110 ... 00000101
                  ^^^^^^^^^      ^^^
                tophash      bucket bits
```

Пусть $B = 3$:

bucket = $101_2 = 5$

tophash = $10110110_2 = 0xB6$

В bucket 5 runtime сначала ищет slots с tophash = 0xB6.

tophash: фильтрация кандидатов

Bucket 5:



Ищем $H_{top} = B6$

Кандидаты только slot 1 и slot 5.

Полное key equality выполняется только для них.

Экономия особенно заметна для строк и сложных comparable-ключей.

Lookup: полный алгоритм

- 1. Посчитать $\text{hash}(\text{key}, \text{hash0})$
- 2. Младшие В бит → выбрать основной bucket
- 3. Старшие 8 бит → вычислить tophash
- 4. Просканировать 8 tophash-байт bucket-a
- 5. Для совпавших tophash выполнить настоящее сравнение key
- 6. Не нашли → перейти в overflow bucket
- 7. Повторять до найденного key или конца цепочки

Коллизии: три уровня

1. Same bucket
Different tophash

2. Same bucket
Same tophash
Different key

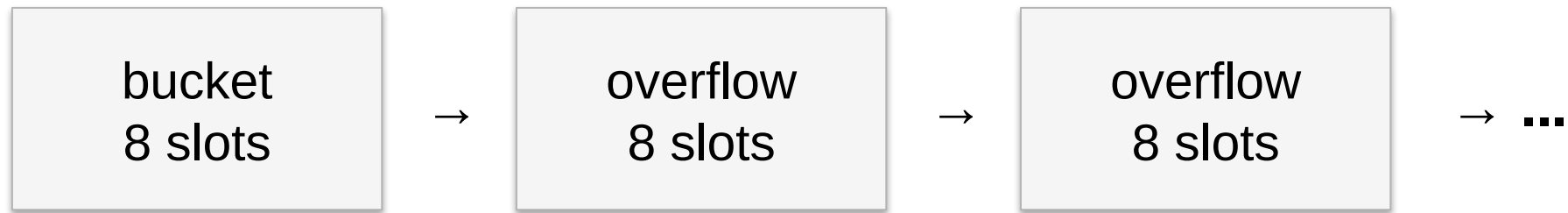
3. Same full hash
Different key

Даже одинаковый hash не означает одинаковый key: окончательное решение всегда делает key equality.

Insert

- Вычислить hash, bucket и tophash
- Просканировать основной и overflow bucket-ы
- Если key уже существует — заменить value
- Если key отсутствует — использовать первый свободный slot
- Если все 8 slots заняты — создать/использовать overflow bucket
- После insert проверить условия grow

Overflow buckets



Цена overflow: pointer chasing, дополнительные cache misses, больше сравнений.

Load factor и порог grow

bucket capacity = 8

grow threshold ≈ 6.5 элемента /
bucket

$\approx 81.25\%$ средней заполненности

Почему не ждать 8/8? При высокой заполненности резко растёт число overflow bucket-ов и стоимость lookup.

Grow: почему нельзя скопировать всё сразу

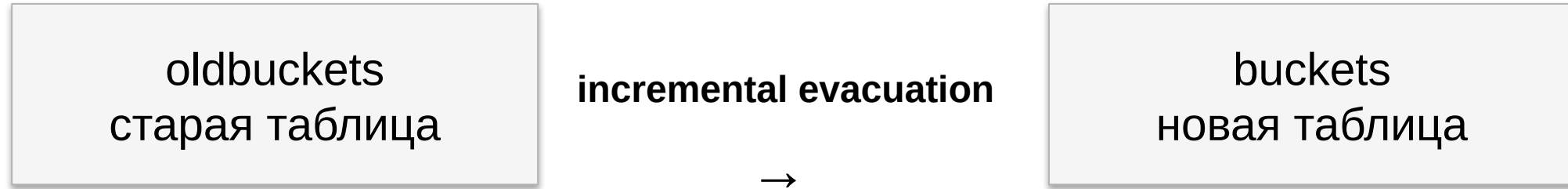
Наивный
вариант:

allocate $\times 2$ → перенести ВСЕ элементы → продолжить

Если map огромная, одна запись получает огромный latency spike.

Go распределяет стоимость grow по последующим операциям.

Grow: oldbuckets и nevacuate



nevacuate = индекс следующего старого bucket, который надо переносить

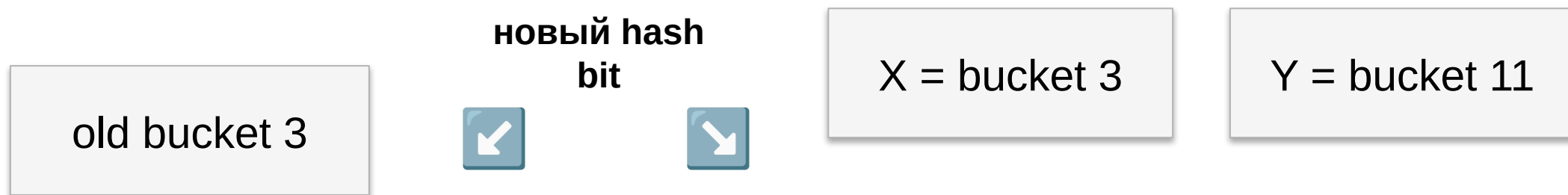
Incremental evacuation

```
write #1 → своя операция + evacuate old bucket 0  
write #2 → своя операция + evacuate old bucket 1  
write #3 → своя операция + evacuate old bucket 2  
...
```

Стоимость роста амортизируется. Нет одной операции, которая обязана полностью перестроить гигантскую map.

X / Y при удвоении таблицы

Было $B = 3 \rightarrow 8$ bucket-ов. После grow $B = 4 \rightarrow 16$ bucket-ов.



Появился один дополнительный значимый bit: он решает, остаётся entry в X или уходит в Y.

Lookup во время grow

Во время grow одновременно живут старая и новая таблицы.

old bucket ещё НЕ evacuated

→ lookup в oldbuckets

old bucket уже evacuated

→ lookup в buckets

Same-size grow

Grow бывает не только $\times 2$.

много overflow bucket-ов
при умеренном `len(m)`

→

новая таблица
того же размера

Смысл: переразложить элементы и убрать накопившийся overflow — фактически compaction.

Delete и память

- `delete(m, key)` уменьшает count и очищает slot
- map не обязана автоматически shrink после удаления
- `len(m) == 0` не означает, что bucket storage снова маленький
- важно для кэшей: большой исторический peak может оставить крупную внутреннюю таблицу
- иногда проще создать новую map и заменить старую

Почему нельзя взять `&m[key]`

Сегодня value может лежать
здесь:

bucket 3 / slot 4

После grow:

bucket 11 / slot 1

**Entry перемещается → стабильный адрес map element нельзя
гарантировать.**

```
u := m[id]  
u.Age++  
m[id] = u
```

```
// либо map[int]*User
```

make(map[K]V, hint)

```
m := make(map[int]User, 100_000)
```

- 100_000 — sizing hint, а не строгая capacity
- runtime заранее выбирает подходящий B
- меньше промежуточных grow
- меньше allocations и evacuation
- полезно, когда размер dataset заранее примерно известен

Map и GC

`map[int]int`

- keys/values без pointers
- GC может меньше сканировать

`map[string]*User`

- pointers внутри layout
- нужны GC pointer metadata

`mapextra`

- помогает удерживать overflow buckets при pointer-free bucket data

Concurrent access

```
go func() { m["a"] = 1 }()  
go func() { m["b"] = 2 }()
```

Запись может менять не только value, но и overflow/grow/evacuation/metadata.

write + write → synchronization required

read + write → synchronization required

fatal error: concurrent map writes — диагностика runtime, а не механизм синхронизации.

Итоговая схема lookup



Что важно запомнить

- Старая Go map = 2^B основных bucket-ов по 8 slots
- low B bits hash выбирают bucket
- tophash — 8-битный fingerprint для дешёвой фильтрации кандидатов
- полное key equality остаётся финальной проверкой
- overflow bucket решает переполнение, но ухудшает locality
- grow выполняется постепенно через oldbuckets и evacuation
- same-size grow убирает чрезмерный overflow
- map element не имеет стабильного адреса
- структура map тесно связана с GC и concurrency